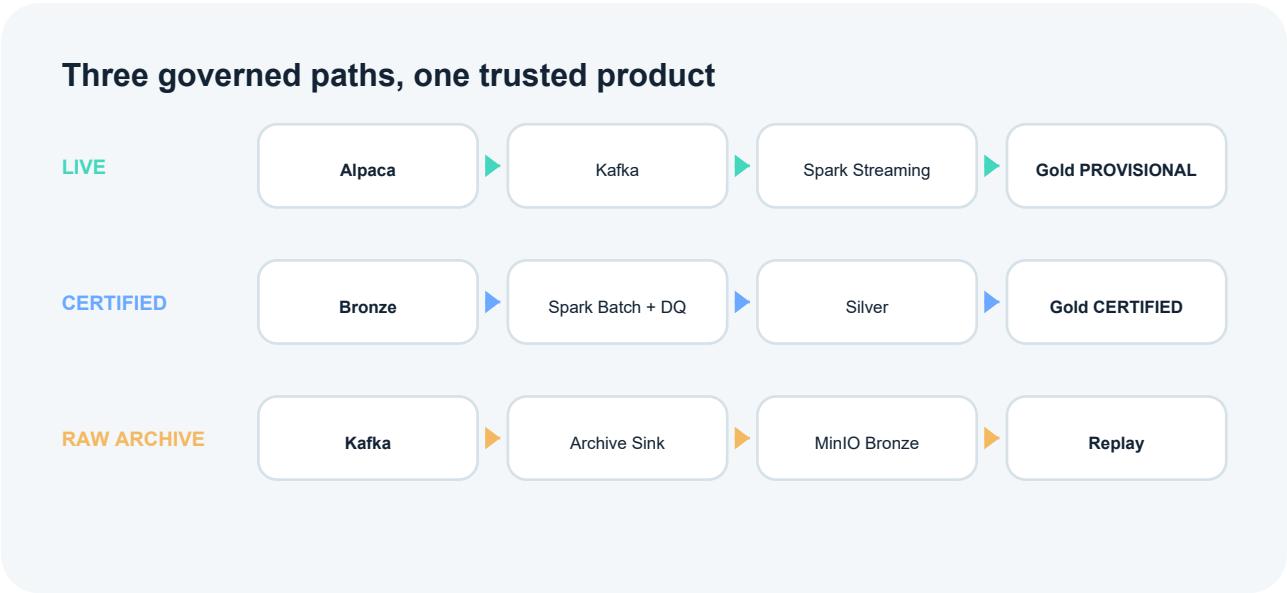


Technical Architecture

Local, reproducible, governed market-data platform

Generated from docs/architecture/architecture.md



1. Architecture summary

MarketPilot separates long-running streaming services from bounded orchestration jobs.

Docker Compose is the service supervisor for the local environment. Airflow is the workflow orchestrator for bounded tasks. Spark provides both Structured Streaming and Batch compute. Kafka decouples ingestion from processing. MinIO provides local object storage. MariaDB is the Gold serving database.

2. Logical layers

Layer	Technology	Responsibility
Sources	Alpaca, SEC EDGAR	External market and filing data
Ingestion	Python services	Connect, normalize event envelope, publish or archive
Transport	Kafka KRaft	Buffering, consumer isolation, replay by offset
Streaming compute	Spark Structured Streaming	Near-real-time validation, indicators, and Gold upserts
Batch compute	Spark Batch	Bronze to Silver and Silver to Gold transformations
Orchestration	Airflow	Schedule and monitor bounded work
Object storage	MinIO, future S3	Bronze, Silver, checkpoints, manifests, archives
Serving storage	MariaDB	Gold tables queried by the application
Application	Backend API, Web App	Secure access and user experience

3. Planned Docker services

Service	Lifecycle	Responsibility
kafka	Long-running	Single KRaft broker for MVP
market-producer	Long-running	Alpaca connection and Kafka publication
raw-archive-sink	Long-running	Kafka to Bronze archive
sec-adapter	Bounded or internal endpoint	SEC polling and raw archive
spark-master	Long-running	Spark standalone cluster coordinator
spark-worker	Long-running	Spark compute resources
spark-streaming	Long-running	Kafka streaming application
Spark Batch applications	Bounded	Bronze to Silver and Silver to Gold
minio	Long-running	Object storage
mariadb	Long-running	Gold business database
airflow-db	Long-running	Airflow PostgreSQL metadata database
airflow-scheduler	Long-running	DAG scheduling
airflow-api-server	Long-running	Airflow UI and API
airflow-dag-processor	Long-running	DAG parsing
backend-api	Long-running	Application data API
web-app	Long-running	Browser UI
operational-monitor	Long-running	Dependency and freshness probes with deduplicated alerts

4. Network boundaries

Recommended logical Docker networks:

- **ingestion-net**: producer, Kafka, raw archive, SEC adapter.
- **processing-net**: Kafka, Spark Master, Spark Worker, streaming and batch jobs.
- **storage-net**: MinIO, MariaDB, Spark, archive services, API.
- **orchestration-net**: Airflow components, Airflow metadata database, Spark Master, SEC adapter.
- **serving-net**: Backend API and Web App.

A service may join more than one internal network where required. Infrastructure ports should not be published to the host unless a developer needs them.

5. Persistent state

Named volumes:

- **kafka-data**
- **minio-data**
- **mariadb-data**

- airflow-db-data
- airflow-logs

Spark checkpoints should be durable across container restarts. For the MVP, validate whether the selected S3A dependencies support MinIO reliably. A named volume is an acceptable fallback for local checkpointing if it is documented and tested.

6. Kafka design

Initial topics:

Topic	Key	Value
market.bars.1m.v1	symbol	Versioned market-bar event
market.bars.1m.backfill.v1	symbol	Historical bars for bounded Bronze certification; not consumed by live streaming
market.bars.1m.dlq.v1	event identifier	Rejected event with reason metadata
sec.filings.v1	accession number	Optional modeled filing event

Initial MVP partitions should be conservative. Increase partition count only after measuring throughput and consumer parallelism. The current event volume does not justify a large local Kafka cluster.

7. Event envelope

Every event should contain:

- event_id
- event_type
- schema_version
- source
- source_event_time
- ingested_at_utc
- correlation_id
- payload

For market bars, the business uniqueness rule is symbol plus event timestamp plus interval.

8. Bronze layout

Bronze is immutable and append-oriented.

Suggested paths:

```
s3a://marketpilot-bronze/source=alpaca/event=market_bar_1m/year=YYYY/month=MM/day=DD/symbol=SYMBOL/
s3a://marketpilot-bronze/source=sec/event=filing/year=YYYY/month=MM/day=DD/form=FORM/
```

Avoid writing one tiny object for every event. Buffer safely and write bounded files while preserving recoverable Kafka offsets and event identifiers.

9. Silver layout

Silver contains canonical schemas, normalized timestamps, typed values, deduplication, validation flags, and symbol linkage.

Suggested path:

```
s3a://marketpilot-silver/dataset=market_bars_1m/year=YYYY/month=MM/day=DD/symbol=SYMBOL/
```

Use Parquet compression and controlled file sizes. The weekly compaction DAG addresses small-file accumulation.

10. Gold database model

Initial MariaDB tables:

Table	Business key	Purpose
<code>dim_symbol</code>	symbol	Asset metadata
<code>fact_market_bar_1m</code>	symbol ID and event UTC timestamp	One-minute OHLCV
<code>fact_indicator_1m</code>	symbol, timestamp, indicator code, version	Technical indicators
<code>fact_signal</code>	symbol, signal timestamp, model version	Application signal
<code>fact_sec_filing</code>	accession number	Filing metadata and Bronze URI
<code>fact_backtest_result</code>	run, symbol, horizon	Backtest results
<code>fact_backtest_run</code>	run ID	Immutable strategy parameters and publication state
<code>fact_backtest_equity_daily</code>	run, symbol, trading date	Bounded application equity curve
<code>etl_watermark</code>	pipeline and partition	Progress and publication state
<code>data_quality_result</code>	run, check, partition	Quality results
<code>archive_manifest</code>	dataset, year, version	Archive verification metadata

Application queries require indexes on symbol and event time. Exact DDL is an implementation task and must be tested against query patterns.

Phase 9 implements four versioned, one-minute Indicator series and sparse, explained Signal observations. A bounded Spark application reads a configured Gold lookback, calculates native Spark window metrics, validates the result, and atomically replaces only the requested UTC date in the two Gold analytics tables. The daily certified DAG invokes it after `silver_to_gold_certified`. Each row retains schema/model, run, code, data, and certification lineage.

Phase 11 adds bounded historical strategy evaluation. Certified Gold bars feed a Spark Batch backtest whose signal from bar `t` is applied only to the following bar return. Full-resolution results are versioned Parquet in MinIO; MariaDB stores run metadata, summary metrics, and a bounded daily equity curve for the API. SPY is a comparison benchmark and no component performs trade execution.

Phase 12 acquires real historical bars without introducing a shortcut around the Medallion pipeline. A manual Airflow task pages through Alpaca IEX, stores each exact API response in content-addressed Bronze objects, normalizes bars to `MarketBarV1`, and publishes them to `market.bars.1m.backfill.v1`. The long-running `raw-archive-sink` consumes both market topics. A bounded barrier confirms the exact topic, partition, and offset object for every published event before the existing Spark Batch certification chain starts. The separate topic

prevents historical traffic from entering the live Structured Streaming application.

11. Streaming application

The Spark Structured Streaming application:

- Reads `market.bars.1m.v1` from Kafka.
- Parses the versioned schema.
- Rejects malformed events to quarantine or DLQ.
- Applies event-time semantics and a watermark.
- May calculate approved near-real-time indicators after a stateful restart and reconciliation design is accepted. Phase 9 keeps the authoritative Indicator calculation in a bounded Spark application.
- Uses `foreachBatch` or another tested sink strategy for transactional MariaDB upserts.
- Persists checkpoints.
- Emits structured operational metrics.

The streaming service must not be implemented as an Airflow task.

12. Airflow architecture

MVP components:

- Airflow Scheduler
- Airflow API Server and UI
- Airflow DAG Processor
- PostgreSQL metadata database
- LocalExecutor

Celery and Redis are intentionally excluded from the MVP.

Airflow submits Spark Batch work to `spark-master` through `SparkSubmitOperator`. Airflow monitors bounded job completion and applies downstream gates.

13. DAG catalogue

DAG	Recommended schedule	Tasks
<code>sec_polling_dag</code>	Every 15 minutes in the configured weekday window	Discover, download, deduplicate, archive, update watermark
<code>daily_market_close_dag</code>	16:30 America/New_York on weekdays	Completeness, Bronze to Silver, Silver DQ, Silver to Gold, Gold DQ, publish
<code>weekly_compaction_dag</code>	Saturday 06:00 America/New_York	Discover small files, compact, validate, manifest
<code>annual_archive_dag</code>	January 10 at 02:00 America/New_York	Export previous year, verify, register manifest
<code>backfill_replay_dag</code>	Manual with validated parameters	Replay selected date range and symbols
<code>historical_backtest_dag</code>	Manual with validated parameters	Validate scope, submit Spark backtest, quality gate, publish
<code>historical_market_backfill</code>	Manual, at most 31 calendar days	Alpaca pages, source archive, Kafka, Bronze barrier, Silver, DQ, Gold Certified, backtest

The daily DAG checks the exchange calendar and short-circuits on non-trading days. A future custom timetable may replace simple cron scheduling.

14. Airflow controls

- `catchup=False` for polling and routine daily DAGs.
- `max_active_runs=1` for partition-mutating DAGs.
- `sec_api_pool=1`.
- `alpaca_api_pool=1` for bounded historical acquisition.
- `spark_batch_pool=1` for the local MVP.
- Two or three retries with exponential backoff.
- Explicit execution timeouts.
- No certified publication after a failed quality gate.
- Parameter validation for backfill and archive ranges.

15. Annual archive

- Select the previous closed calendar year.
- Export eligible MariaDB partitions to versioned Parquet paths.
- Record schema version and code version.
- Validate row count, min and max timestamp, checksum, and sample queries.
- Register `archive_manifest` as verified.
- Do not automatically purge MariaDB in the MVP.

The implementation writes immutable, versioned Parquet objects to the dedicated `marketpilot-archive` bucket. Every object has a SHA-256 digest; the deterministic inventory digest, counts, time bounds, run ID, code version, and closed-period flag are stored in both an object-store manifest and `archive_manifest`. Restore drills first verify the complete inventory and write only to an isolated restore schema.

Weekly Silver compaction follows the same safety model: measure input rows, business keys, logical row hashes, and schema; write staging output; back up the original objects; replace; then publish its manifest. A failed replacement restores the run-specific backup.

16. API boundary

The Backend API reads Gold tables through a narrowly scoped database identity. It provides pagination, filtering, bounded date ranges, input validation, and safe response models.

The Web App has no database credentials and no MinIO credentials.

The Phase 9 `/api/v1/indicators` and `/api/v1/signals` endpoints use the same symbol validation, maximum 31-day range, parameterized SQL, and pagination ceiling as market bars. Explanations contain calculated market context only; internal lineage and credentials remain outside browser response models.

The implemented Phase 7 boundary uses a `marketpilot_app` identity with `SELECT` only on symbols, market bars, SEC filings, and watermarks. Market queries default to seven days and cannot exceed 31 days; all collection responses enforce a page and page-size ceiling. Internal Bronze URIs and lineage fields are not part of the browser response model.

The Nginx Web App serves static assets and proxies relative `/api/` paths to the Backend API over `serving-plane`. This keeps database addressing and credentials out of browser code while allowing the API container to be restarted independently.

17. Observability

Minimum metrics:

- producer connection status and reconnect count;
- Kafka publish failures;
- Kafka consumer lag;
- source-to-Gold freshness;
- Spark batch and streaming durations;
- checkpoint progress;
- expected versus actual bars by symbol and session;
- duplicate and rejected event counts;
- Airflow DAG status and duration;
- MariaDB connection and query latency;
- object count and small-file growth.

The local `operational-monitor` is supervised by Compose, not Airflow. It performs read-only dependency and freshness probes, writes structured JSON logs, and emits only state transitions to avoid repeated alerts. A generic webhook is optional and disabled by default.

18. Decision Intelligence

Phase 14 adds an application-facing decision-support layer without changing the source-of-truth paths. Certified bars are resampled into 5-minute, 15-minute, hourly and daily feature windows; SEC Company Facts are archived raw, normalized and published as versioned Gold fundamentals. Corporate actions are fetched by a bounded Airflow DAG, retained as exact content-addressed Bronze pages and normalized in Gold. Split factors are applied to analytical copies before return and level calculations; immutable raw bars are never rewritten. Cash-dividend events are retained for audit and future total-return models, while the current price-signal backtest does not credit cash distributions.

The live Structured Streaming path may publish **PROVISIONAL** snapshots only after a closed 15-minute window. The bounded post-market Spark job publishes authoritative **CERTIFIED** snapshots after market, fundamental and quality gates. Each snapshot includes its input window, feed, formula/model version, lineage, expiry and explanation. The first 20 sessions are Shadow Mode and therefore never actionable. See ADR-008.

After every certified daily publication, a bounded evaluator measures recommendation outcomes at 2, 5, 10 and 20 XNYS sessions. The conservative path rule records a stop when an aggregated path touches both a stop and a target and intrabar ordering is unknown. Promotion status advances only to **READY_FOR_REVIEW**; software never self-approves the model.

In-app alerts are append-only transition events (**ENTERED_BUY_ZONE**, **ACTION_CHANGED**, stale-data or risk warnings). They are informational only and cannot trigger an order. The Shadow Mode gate, evaluation summary and alerts are exposed through bounded read-only API endpoints.

The browser reads recommendations only through the Backend API. Manual watchlist and portfolio writes require a separate, least-privilege identity; the existing market-data API identity remains read-only. The **marketpilot_user_write** identity can modify only the local portfolio, position and watchlist tables and can read only symbol identifiers needed for validation. Browser payloads are schema-bounded and symbol-validated before a transaction replaces the saved watchlist. No component submits an order or reads an Alpaca brokerage balance.

19. Capacity assumptions

The MVP workload is small, but the local stack is resource-heavy. A practical workstation target is:

- 8 CPU cores preferred;
- 16GB RAM minimum for comfortable concurrent operation;
- 100GB SSD preferred;
- explicit container memory limits after empirical testing.

Do not allocate all host memory to Spark. Leave capacity for Docker, Kafka, Airflow, MariaDB, MinIO, the API, and the OS.